

**1. Что делает BeanFactoryPostProcessor?**

- A) Меняет поля бина после инициализации
- B) Позволяет изменять BeanDefinition до создания бинов**
- C) Перехватывает вызовы методов
- D) Обработывает HTTP-запросы

**Пояснение:**

BeanFactoryPostProcessor работает **до** того, как создаются экземпляры бинов, и позволяет модифицировать их метаданные (описания).

**2. Что будет выведено?**

```
@Component
class MyBPP implements BeanPostProcessor {
    public Object postProcessBeforeInitialization(Object bean, String name) {
        if (bean instanceof DataSource) System.out.println("Before: " + name);
        return bean;
    }
}
```

- A) Ошибка
- B) Выведет "Before: dataSource" при создании DataSource**
- C) Ничего
- D) NullPointerException

**Пояснение:**

Метод postProcessBeforeInitialization сработает перед инициализацией каждого подходящего бина.

**3. Как можно изменить значение поля бина до его инициализации, не редактируя сам класс?**

- A) Через @Autowired
- B) Через BeanPostProcessor с reflection**
- C) Через ApplicationListener
- D) Через XML

**Пояснение:**

BeanPostProcessor позволяет модифицировать созданный бин (включая приватные поля).

**4. Когда срабатывает PropertySourcesPlaceholderConfigurer?**

- A) После всех BeanPostProcessor
- B) При старте приложения
- C) До инициализации бинов, как часть BeanFactoryPostProcessor**
- D) После вызова run()

**Пояснение:**

Он реализует BeanFactoryPostProcessor, значит, выполняется *до создания бинов*.

**5. Можно ли в BeanFactoryPostProcessor получить доступ к конкретному экземпляру бина?**

- A) Да
- B) Нет, бины ещё не созданы**
- C) Только через reflection
- D) Только если singleton

**Пояснение:**

BeanFactoryPostProcessor работает с метаданными (BeanDefinition), а не с самими объектами.